

Go 语言程序设计第二次作业：基于接口的 AI 迭代开发

学号：2024131065

姓名：覃冠三

第一版 (V1): 初版代码

开发记录：在最初的提示词中，我要求 AI “生成一个包含三种基础密码学处理（翻转、MD5、SHA256）的程序”。AI 最初给出的方案是将所有逻辑写在 `main` 函数的 `switch-case` 中。我拒绝了该方案，并手动重构了代码结构，定义了统一的 `CryptoTask` 接口，要求后续所有功能必须基于该接口实现。

```
package main

import (
    "crypto/md5"
    "crypto/sha256"
    "fmt"
)

// 1. 定义统一的接口，规范输入输出
type CryptoTask interface {
    Name() string
    Execute(data string) string
}

// 2. 基础功能 1：字符串反转（模拟简单的换位密码）
type ReverseTask struct{}

func (t ReverseTask) Name() string { return "String Reverse" }

func (t ReverseTask) Execute(data string) string {
    runes := []rune(data)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        runes[i], runes[j] = runes[j], runes[i]
    }
}
```

```

    return string(runes)
}
// 3. 基础功能 2: MD5 哈希计算
type MD5Task struct{}
func (t MD5Task) Name() string { return "MD5 Hash" }
func (t MD5Task) Execute(data string) string {
    hash := md5.Sum([]byte(data))
    return fmt.Sprintf("%x", hash)
}
// 4. 基础功能 3: SHA-256 哈希计算
type SHA256Task struct{}
func (t SHA256Task) Name() string { return "SHA-256 Hash" }
func (t SHA256Task) Execute(data string) string {
    hash := sha256.Sum256([]byte(data))
    return fmt.Sprintf("%x", hash)
}
func main() {
    input := "hello blockchain"
    // 接口的优势: 可以通过切片统一管理不同类型的结构体实例
    tasks := []CryptoTask{
        ReverseTask{},
        MD5Task{},
        SHA256Task{},
    }
    fmt.Println("--- V1: 接口化按序执行任务 ---")
    for _, task := range tasks {
        result := task.Execute(input)
        fmt.Printf("[%s] Output: %s\n", task.Name(), result)
    }
}

```

```
}  
  
}
```

```
main.go X  
assignment02_crypto > v1 > main.go > ...  
1  package main  
2  
3  import (  
4      "crypto/md5"  
5      "crypto/sha256"  
6      "fmt"  
7  )  
8  
9  // 1. 定义统一的接口  
10 type CryptoTask interface {  
11     Name() string  
12     Execute(data string) string  
13 }  
14  
15 // 2. 实现具体功能: 字符串反转  
16 type ReverseTask struct{}  
17  
18 func (t ReverseTask) Name() string { return "String Reverse" }  
19 func (t ReverseTask) Execute(data string) string {  
20     runes := []rune(data)  
21     for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {  
22         runes[i], runes[j] = runes[j], runes[i]  
23     }  
24     return string(runes)  
25 }  
26  
27 // 3. 实现具体功能: MD5摘要  
28 type MD5Task struct{}  
29  
30 func (t MD5Task) Name() string { return "MD5 Hash" }  
31 func (t MD5Task) Execute(data string) string {  
32     hash := md5.Sum([]byte(data))  
33     return fmt.Sprintf("%x", hash)  
34 }  
35  
36 // 4. 实现具体功能: SHA256摘要  
37 type SHA256Task struct{}  
38  
39 func (t SHA256Task) Name() string { return "SHA-256 Hash" }  
40 func (t SHA256Task) Execute(data string) string {  
41     hash := sha256.Sum256([]byte(data))  
42     return fmt.Sprintf("%x", hash)
```

第二版 (V2): 迭代代码

开发记录: 我向 AI 提出追加 Base64 和 Hex 十六进制编码的功能。由于 V1 底层的接口框架已经定型, AI 顺理成章地生成了两个新的结构体并实现了相应的方法。主函数 main 中的调度逻辑一行都没有修改, 只需将新对象放入切片即可。

```
package main

import (
    "crypto/md5"
    "crypto/sha256"
    "encoding/base64"
    "encoding/hex"
    "fmt"
)

type CryptoTask interface {
    Name() string
    Execute(data string) string
}

// ... (此处省略 V1 中的 ReverseTask, MD5Task, SHA256Task 代码, 与 V1 完全一致) ...

// 【新增功能 1】: Base64 编码
type Base64Task struct{}

func (t Base64Task) Name() string { return "Base64 Encode" }

func (t Base64Task) Execute(data string) string {
    return base64.StdEncoding.EncodeToString([]byte(data))
}

// 【新增功能 2】: Hex 十六进制编码
type HexTask struct{}

func (t HexTask) Name() string { return "Hex Encode" }

func (t HexTask) Execute(data string) string {
    return hex.EncodeToString([]byte(data))
}
```

```
}  
func main() {  
    input := "hello blockchain"  
    // 新增功能只需在此处追加，符合开闭原则（OCP）  
    tasks := []CryptoTask{  
        ReverseTask{},  
        MD5Task{},  
        SHA256Task{},  
        Base64Task{},  
        HexTask{},  
    }  
    fmt.Println("--- V2: 迭代追加编码功能 ---")  
    for _, task := range tasks {  
        result := task.Execute(input)  
        fmt.Printf("[%s] Output: %s\n", task.Name(), result)  
    }  
}
```

main.go X

assignment02_crypto > v2 > main.go > ...

```
35  type SHA256Task struct{}
36
37  func (t SHA256Task) Name() string { return "SHA-256 Hash" }
38  func (t SHA256Task) Execute(data string) string {
39      hash := sha256.Sum256([]byte(data))
40      return fmt.Sprintf("%x", hash)
41  }
42
43  // 【新增功能 1】: Base64编码
44  type Base64Task struct{}
45
46  func (t Base64Task) Name() string { return "Base64 Encode" }
47  func (t Base64Task) Execute(data string) string {
48      return base64.StdEncoding.EncodeToString([]byte(data))
49  }
50
51  // 【新增功能 2】: Hex十六进制编码
52  type HexTask struct{}
53
54  func (t HexTask) Name() string { return "Hex Encode" }
55  func (t HexTask) Execute(data string) string {
56      return hex.EncodeToString([]byte(data))
57  }
58
59  func main() {
60      input := "hello blockchain"
61
62      // 直接在任务列表中追加，无需修改调用逻辑
63      tasks := []CryptoTask{
64          ReverseTask{},
65          MD5Task{},
66          SHA256Task{},
67          Base64Task{},
68          HexTask{},
69      }
70
71      fmt.Println("--- v2: 迭代新增任务 ---")
72      for _, task := range tasks {
73          result := task.Execute(input)
74          fmt.Printf("[%s] Output: %s\n", task.Name(), result)
75      }
76  }
```

第三版 (V3): 优化代码

开发记录: 按照要求, 我选取了两个耗时潜力最大的功能 (MD5 和 SHA256) 加入并发。AI 最初生成的代码使用了 `time.Sleep()` 来等待协程结束, 这在工程上是极度不严谨的。我接管了代码, 引入了 `sync.WaitGroup` 来进行精确的并发控制, 重构了协程的调度逻辑。

```
package main

import (
    "crypto/md5"
    "crypto/sha256"
    "fmt"
    "sync"
)

type CryptoTask interface {
    Name() string
    Execute(data string) string
}

// ... (省略具体 Task 的实现, 保持不变) ...

func main() {
    input := "hello blockchain"

    fmt.Println("--- V3: 并发执行计算密集型任务 ---")

    // 任选两个功能: 实例化我们要并发执行的任务
    task1 := MD5Task{}
    task2 := SHA256Task{}

    // 引入 WaitGroup 保证主协程等待子协程完毕
    var wg sync.WaitGroup

    // 启动第一个 goroutine
    wg.Add(1)

    go func(t CryptoTask) {
```

```
    defer wg.Done()

    fmt.Printf("[Goroutine 1] %s Result: %s\n", t.Name(), t.Execute(input))
}(task1)

// 启动第二个 goroutine
wg.Add(1)

go func(t CryptoTask) {
    defer wg.Done()

    fmt.Printf("[Goroutine 2] %s Result: %s\n", t.Name(), t.Execute(input))
}(task2)

// 阻塞等待
wg.Wait()

fmt.Println("所有并发协程执行完毕。")
}
```



```
main.go X
assignment02_crypto > v3 > main.go > ...

45
46 func (t Base64Task) Name() string { return "Base64 Encode" }
47 func (t Base64Task) Execute(data string) string {
48     return base64.StdEncoding.EncodeToString([]byte(data))
49 }
50
51 type HexTask struct{}
52
53 func (t HexTask) Name() string { return "Hex Encode" }
54 func (t HexTask) Execute(data string) string {
55     return hex.EncodeToString([]byte(data))
56 }
57
58 func main() {
59     input := "hello blockchain"
60
61     // 并发执行任务集
62     concurrentTasks := []CryptoTask{
63         MD5Task{},
64         SHA256Task{},
65     }
66
67     fmt.Println("--- V3: 并发执行优化 ---")
68
69     var wg sync.WaitGroup
70
71     for _, task := range concurrentTasks {
72         wg.Add(1)
73         // 闭包中传递 task 以避免循环变量捕获问题
74         go func(t CryptoTask) {
75             defer wg.Done()
76             result := t.Execute(input)
77             // 注意: 并发输出的顺序是不可预测的
78             fmt.Printf("[Concurrent - %s] Output: %s\n", t.Name(), result)
79         }(task)
80     }
81
82     wg.Wait() // 阻塞主协程, 直到所有子协程完成
83     fmt.Println("所有并发任务执行完毕。")
84 }
85
```

实验反思与开发心得

1. V1 到 V2 过程中，你增加了什么功能？修改是否顺利？

我增加了 Base64 和 Hex 两种数据编码功能。整个迭代过程非常顺利。因为我在 V1 中确立了 `CryptoTask` 接口，V2 的修改变成了单纯的做加法。我不需要去改动原来的哈希计算代码，也不需要去改动 `main` 函数里循环执行的底层逻辑，只需新建两个结构体实现 `Name()` 和 `Execute()`，完美具象化了软件工程中的开闭原则。

2. V2 到 V3 过程中，你做了哪些优化？

我将原本串行执行的 MD5 和 SHA256 哈希计算放入了单独的 `goroutine` 中并发执行。在结构优化上，我清理了多余的循环，并引入了 `sync.WaitGroup` 来替代 AI 最初生成的粗暴的 `time.Sleep()`，确保主程序能精准捕捉到子协程完成的信号，提高了代码的健壮性和规范性。

3. 接口在这次作业中起到了什么作用？

接口起到了**强制契约**和**逻辑解耦**的作用。如果没有接口，不同功能的调用势必需要写死成 `taskA(data)`、`taskB(data)`，或者使用臃肿的 `if-else / switch` 语句来判断任务类型。引入接口后，不论底层是简单的字符串翻转还是复杂的哈希运算，在主函数看来它们统统只是一个 `CryptoTask` 对象。接口统一了不同行为的调用入口，使得主程序的视野变高了，不再关心具体的实现细节。

4. 你亲自修改了哪些代码，为什么要改？

我主要接管并修改了两处 AI 生成的代码：

- **架构重构：**AI 最初倾向于写流水账式的过程式代码，我强制自己手搭了基于 `interface` 的骨架。
- **并发控制逻辑：**在 V3 加入并发时，AI 给出的第一版方案不仅用了不可靠的 `time.Sleep()` 来等待协程，甚至在尝试用 `for` 循环开协程时，直接捕获了外部的循环变量。在 Go 语言中，这是一个经典的**闭包变量捕获陷阱**，会导致所有协程最后打印出来的都是同一个任务的结果。我修改了这部分逻辑，改为通过传参 `go func(t CryptoTask)` 的形式将变量安全拷贝，并使用了标准的 `sync.WaitGroup` 同步原语。必须严谨对待并发，不能得过且过。

5. `goroutine` 加入后，程序运行方式有什么变化？

程序的运行机制发生了本质改变：**机会与风险并存**。

原本程序是“阻塞且确定”的，一行执行完才执行下一行，输出顺序永远固定。加入 `goroutine` 后，程序变成了“异步且非确定”的。MD5 和 SHA256 的协程被交由 Go 的 GMP 调度器分配到不同的系统线程上并行计算，显著提高了运行效率。

但这也意味着，每次运行结果输出的先后顺序是未知的，如果不加锁或使用通道进行状态同步，极易在共享资源时引发数据竞争。

6. 这次作业中，你觉得 AI 最有帮助的地方是什么？最不可靠的地方是什么？

- **最有帮助的地方：**充当了高级的“API 词典”和“打字机”。对于 `crypto/sha256`、`encoding/base64` 等标准库的具体包名和调用姿势，AI 能瞬间写出准确的样板代码，省去了我大量查阅文档的低效时间。
- **最不可靠的地方：**缺乏全局架构观和严谨的系统思维。AI 往往为了追求程序能跑通而牺牲代码质量，比如提供有隐患的并发代码、忽略错误处理。这让我深刻认识到，面对关键的决策性逻辑，必须认真严格审查。